

Threads (II)

Lezione 16

Sistemi Operativi - II Modulo (Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

- Richiami su:
 - esecuzione concorrente;
 - race condition;
 - regione critica;
 - semafori
- Meccanismi di sincronizzazione dei thread in linux (POSIX)
 - Mutex (semaforo binario)
 - Barrier (barriera)
 - Condition (condizione)



Richiami

esecuzione concorrente; race condition; regione critica; semafori



Flussi di esecuzione concorrente

- La fonte di maggior complicazione nei sistemi operativi è costituita dall'esistenza di flussi di esecuzione concorrenti:
 - In una **applicazione multithread**, ciascun thread è un flusso di esecuzione “in competizione” con quelli degli altri thread
 - In un sistema multiprogrammato con prelazione, ciascun processo può essere interrotto da un altro processo
 - In un sistema multiprocessore, diversi processi o gestori di interruzione sono in esecuzione contemporaneamente
- La coerenza delle **strutture di dati private** di ciascun flusso di esecuzione è garantita dal meccanismo di cambio di flusso (contesto)
- La coerenza delle **strutture di dati condivise** tra i vari flussi di esecuzione concorrenti **non è garantita** da tale meccanismo



Race condition

- Se due o più flussi di esecuzione hanno una struttura dati condivisa, la concorrenza dei flussi può determinare uno stato della struttura non coerente con la logica di ciascuno dei flussi
- *Race condition*
 - Lo stato della memoria condivisa tra due o più flussi di esecuzione concorrenti dipende dall'ordine esatto (temporizzazione) degli accessi alla memoria stessa
- Le race conditions sono tra gli errori del software più insidiosi e difficili da determinare:
 - non sono per loro natura deterministici
 - possono non apparire mai sul sistema di sviluppo
 - possono non essere facilmente replicabili
 - sono di difficile comprensione

Esempio race condition

- Flusso #1 e Flusso #2 condividono una variabile `counter`
- L'incremento e decremento di `counter` non sono operazioni *atomiche* (indivisibili)!
- Ciascuna consiste nel:
 - Trasferimento del valore di `counter` in un registro (load)
 - Operazione di incremento o decremento sul registro
 - Trasferimento del valore del registro in `counter` (store)

Flusso #1:

```
for (;;) {  
    crea_risorsa();  
    counter = counter + 1;  
}
```

Flusso #2:

```
while (counter > 0) {  
    counter = counter - 1;  
    consuma_risorsa();  
}
```

Esempio race condition (cont.)

- $t=1$ counter = 2
- $t=6$ counter = 2 **OK**

Istante	Flusso #1	Flusso #2
1	$R_0 \leftarrow \text{counter}$ (2)	
2	$R_0 \leftarrow R_0 + 1$ (3)	
3	$\text{counter} \leftarrow R_0$ (3)	
4		$R_1 \leftarrow \text{counter}$ (3)
5		$R_1 \leftarrow R_1 - 1$ (2)
6		$\text{counter} \leftarrow R_1$ (2)

- $t=1$ counter = 2
- $t=6$ counter = 3 **KO**

Istante	Flusso #1	Flusso #2
1	$R_0 \leftarrow \text{counter}$ (2)	
2	$R_0 \leftarrow R_0 + 1$ (3)	
3		$R_1 \leftarrow \text{counter}$ (2)
4		$R_1 \leftarrow R_1 - 1$ (1)
5		$\text{counter} \leftarrow R_1$ (1)
6	$\text{counter} \leftarrow R_0$ (3)	

Regione critica

- Una sequenza di istruzioni di un flusso di esecuzione che accede ad una struttura di dati condivisa e che quindi non deve essere eseguita in modo concorrente ad un'altra regione critica
- Per realizzare una regione critica, ovvero per avere l'accesso esclusivo ad una risorse condivisa si può usare un **semaforo**

Semaforo

- Inventato da E. W. Dijkstra negli anni '60
- Basato su una variabile intera *contatore*
 - inizializzata con valore $n > 0$
 - Accessibile tramite due primitive *atomiche*:
 - *Wait*: attende che il contatore sia positivo, poi lo decrementa
 - *Signal*: incrementa il contatore
- Semaforo contatore
 - $n > 1$ può essere acquisito da n flussi di esecuzione in modo concorrente
- Semaforo binario (**Mutex - Mutual exclusion**)
 - $n = 1$ può essere acquisito da un solo flusso alla volta

Mutex POSIX



POSIX Mutex (MUTual EXclusion device)

- Utile per proteggere strutture dati condivise e realizzare regioni critiche
- Li vediamo nell'ambito thread
 - `int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *mutexattr);`
 - `int pthread_mutex_lock(pthread_mutex_t *mutex);`
 - `int pthread_mutex_trylock(pthread_mutex_t *mutex);`
 - `int pthread_mutex_unlock(pthread_mutex_t *mutex);`
 - `int pthread_mutex_destroy(pthread_mutex_t *mutex);`
- `man pthread_mutex_*`
- se non installata, su Debian `sudo apt install glibc-doc`



```
int pthread_mutex_init(pthread_mutex_t *mutex,  
const pthread_mutexattr_t *mutexattr);
```

- Inizializza un mutex e imposta gli attributi a mutexattr
- Gli attributi determinano il comportamento del semaforo quando il thread invoca un lock/unlock più volte consecutivamente
- Se mutexattr=NULL viene impostato il valore di default (`fast`) altrimenti può essere specificato `recursive` o `error checking`
 - `fast`
 - un lock blocca il thread finché il lock precedente non è rilasciato (può creare **stallo**);
 - unlock rilascia il semaforo e ritorna subito
 - `recursive`
 - permette allo stesso thread di mettere più lock (un contatore tiene conto del numero di lock messi);
 - unlock decrementa il contatore; rilascia il semaforo quando il contatore è 0
 - `error checking`
 - genera un errore in caso il thread cerchi di mettere un lock su di un mutex sul quale già detiene un lock;
 - unlock ritorna errore se il thread non aveva fatto una lock precedentemente

Esempio



S,E

- produttore - consumatore su buffer circolare condiviso
- un processo (thread) produttore mette un carattere in un buffer circolare condiviso
- uno o più processi (thread) consumatori estraggono il contenuto del buffer
- vedi `prodcons_thread.c`
- Esercizio: modificare il programma `prodcons_thread.c` in modo che nel buffer venga anche scritto l'identificatore del thread che ha scritto il carattere nel buffer. Aggiungere un altro thread produttore.

Sincronizzazione thread: barriera



Barriera

- Metodo di sincronizzazione tra processi o thread
- Un insieme di thread continua il proprio flusso di esecuzione solo se tutti hanno raggiunto la barriera

```
int pthread_barrier_init(pthread_barrier_t * restrict  
barrier, const pthread_barrierattr_t * restrict attr,  
unsigned int count);
```

```
int pthread_barrier_destroy(pthread_barrier_t *barrier);
```

```
int pthread_barrier_wait(pthread_barrier_t *barrier);
```

- man pthread_barrier_*
- [Debian/Ubuntu sudo apt install manpages-posix
manpages-posix-dev]



```
int pthread_barrier_init(pthread_barrier_t * restrict  
barrier, const pthread_barrierattr_t * restrict attr,  
unsigned int count);
```

- Crea una nuova barriera con attributi `attr` e per `count` thread (ovvero `count` threads parteciperanno alla barriera)
- Se `attr = NULL` viene impostato l'attributo di default
 - noi useremo il default
- `pthread_barrier_wait` - Se la chiamata avviene con successo restituisce
 - `PTHREAD_BARRIER_SERIAL_THREAD` ad un thread in attesa sulla barrier selezionato casualmente
 - 0 a tutti gli altri thread in attesa sulla barriera
- vediamo un esempio di uso delle barriere `thread_barrier.c`

Conditions



Condition

- Permette ad un thread di sospendere la sua esecuzione finché un predicato su di un dato condiviso non è verificato
 - `int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *cond_attr);`
 - `int pthread_cond_signal(pthread_cond_t *cond);`
 - `int pthread_cond_broadcast(pthread_cond_t *cond);`
 - `int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);`
 - `int pthread_cond_timedwait(pthread_cond_t *cond, pthread_mutex_t *mutex, const struct timespec *abstime);`
 - `int pthread_cond_destroy(pthread_cond_t *cond);`
- Una variabile condition deve essere sempre associata ad un mutex per evitare una race condition dove un thread è in attesa di una condizione ed un altro thread invia un segnale alla condizione (sblocca la condizione) prima che il primo thread si metta in attesa

Condition (cont.)

- `int pthread_cond_signal(pthread_cond_t *cond);`
 - risveglia uno dei thread in attesa sulla condizione
- `int pthread_cond_broadcast(pthread_cond_t *cond);`
 - risveglia tutti i thread in attesa sulla condizione
- `int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);`
 - Sblocca il mutex associato alla condizione e attende che un altro thread esegua un `pthread_cond_signal` sulla condizione
 - Il mutex va prima bloccato (lock) dal thread che invoca la `wait`
 - Prima di terminare, **wait** rimette il lock sul mutex
- Vedere l'esempio `prodcons_cond.c`

```
lock mutex
wait cond (implicit unlock
/ lock )
/*sezione critica*/
unlock mutex
```



Esercizi

- Nella lezione precedente erano stati assegnati 6 esercizi. Considerate ora gli Esercizi ES5 e ES6 alla luce della sincronizzazione tra thread ed in particolare all'uso dei mutex e delle condition.
- Quindi svolgete i seguenti esercizi
- ES1: riscrivere l'esercizio ES5 usando un `pthread_mutex`
- ES2: riscrivere l'esercizio ES6 usando un `pthread_mutex`
- ES4: riscrivere l'esercizio ES6 usando un `pthread_cond`

Domande?

